- Compute Match — GPU Capacity Model and CSV Import Workflow
  - GPU offering data model
  - CSV capacity import — end-to-end flow
  - CSV columns
  - NCP onboarding CSV
  - Legacy bootstrap importers

Compute Match — GPU Capacity Model and CSV Import Workflow

GPU offering data model

A **GPU offering** (`backend/core/gpu_offering/gpu_offering.py`) represents a GPU cluster type an NCP offers:

- GPU model / platform (for example H100, H200, B200, GB200)
- Provider organization and **hosting provider** (the NCP's data-center brand)
- Description, product link (must be HTTP(S)), VRAM (positive if set)
- Interconnect: `InfiniBand` or `Spectrum-X`
- Status: `ACTIVE` (listed to customers), `PENDING_REVIEW` (awaiting admin review), `REJECTED`
- One or more **regional capacities**

A **regional capacity** ties the offering to a location: geographical region, country, address and latitude/longitude, with a list of **capacity windows**.

A **capacity window** has a date range (finite or open-ended), GPU count, delivery id, block location, a **capacity type** (`AVAILABLE` or `FUTURE`) and an AI-compute flag.

Reference data: - **Region** — a data-center location belonging to a hosting provider and a geographical region. - **Geographical region** — a broad area with a code, display name and list of ISO country codes. - **GPU model** — catalog of platform names.

CSV capacity import — end-to-end flow

1. **Upload.** An NCP with `manage_inventory` uploads a CSV on **Upload Capacities** (`POST /api/v1/provider/gpu-offerings/import` , returns 202 Accepted). The backend checks size (10 MB) and row count (10,000).
2. **Job created.** An `ImportJob` is stored with status `QUEUED` , and Temporal workflow `CsvImportWorkflow` starts with id `csv-import-{job_id}` .
3. **Validate.** The worker validates columns and rows (`VALIDATING`). If nothing is valid → `VALIDATION_FAILED` .
4. **Import.** Valid rows are imported atomically (`IMPORTING`). Failures here → `IMPORT_FAILED` .
5. **Classify.** Rows starting within 90 days of upload become `AVAILABLE` and are published immediately. Rows starting later become `FUTURE` .
6. **Review (future only).** If there are new future rows, the job moves to `PENDING_REVIEW` , admins are emailed, and the workflow waits for an `approve_import` or `reject_import` signal.
7. **Decision.** A Platform Admin with `review_capacity` accepts (`APPROVED` then `COMPLETED` , offerings activated) or rejects (`REJECTED` / `FUTURE_CAPACITY_REJECTED` , with reason).
8. **Notify.** The NCP is emailed the result.

Import job statuses

`QUEUED` , `VALIDATING` , `VALIDATION_FAILED` , `IMPORTING` , `IMPORT_FAILED` , `PENDING_REVIEW` , `APPROVED` , `REJECTED` , `FUTURE_CAPACITY_REJECTED` , `COMPLETED` .

UI labels: Processing, Pending Review, Completed, Rejected, Validation Failed.

Retry behaviour

- Import activities retry up to 3 times; validation and CSV-format errors are not retried.
- The workflow itself runs at most once, so users never get duplicate emails from replays.

CSV columns

Column	Required	Notes
GPU Platform Type	Yes	Vendor prefix such as “NVIDIA” tolerated
Number of GPUs Available	Yes	Positive integer

Column	Required	Notes
Live Token Date	Yes	Start of availability; decides AVAILABLE vs FUTURE
Country	Yes	Names normalized (“USA” → United States, “Holland” → Netherlands)
City	Yes	Matched against a city index with alternate names
Region	No	
end_datetime	No	End of availability; open-ended if blank
NCP Name, RA/RD, Exemplar Cluster, aaS Capability, State	No	Metadata, ignored on import

Limits: UTF-8, 10 MB, 10,000 rows, 500 characters per cell. Formula-like cells (`=` , `+` , `-` , `@`) are neutralized.

Note: the in-app User Manual still says 1,000 rows; the backend enforces 10,000 (`MAX_CSV_ROWS` in `backend/core/gpu_offering/gpu_offering_csv_import_fields.py`).

NCP onboarding CSV

Admins with `manage_ncp` upload an onboarding CSV (`POST /api/v1/provider/onboard-providers`). `ProviderOnboardingWorkflow` validates it, creates Keycloak organizations, users, roles and group memberships (30-minute timeout), then emails the initiator. Statuses: `QUEUED` , `VALIDATING` , `VALIDATION_FAILED` , `ONBOARDING` , `COMPLETED` , `FAILED` .

Legacy bootstrap importers

For seeding environments there are offline scripts in `backend/` : `import_excel_gpu_data.py` (see `EXCEL_IMPORT_README.md`), `csv_data_importer.py` , `csv_gpu_importer.py` , `generate_seed_data.py` . They use an older column layout and are separate from the production Temporal import path.